



南方科技大学

LAB 6: Full Transformer Architecture

COE 201 - AI Large Models, Spring 2026

2026-03-30

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Recap: What We've Built So Far	1
Positional Encoding	3
Residual Connections & Layer Normalization	13
Cross-Attention	20
Transformer Blocks	24
Lab 6 Exercises	31

Recap: What We've Built So Far

Review Lab 5 achievements and identify the remaining building blocks for a complete Transformer.

What We've Built So Far



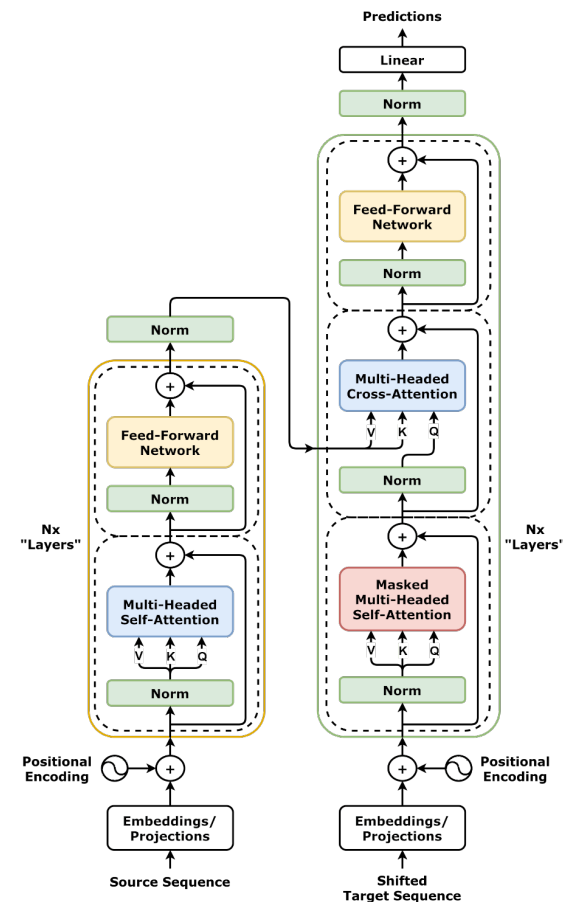
南方科技大学

✓ Already Done in Lab Course

- Scaled Dot-Product Attention
- Multi-Head Mechanism
- Causal Masking

⚠ What is still missing? (Lab 6)

- Positional Encoding
- Residual & LayerNorm
- Cross-Attention
- Full Transformer Blocks



Positional Encoding

Inject sequence order into permutation-invariant attention via sinusoidal functions.

The Problem: Attention is Order-Agnostic



南方科技大学

Self-Attention is **permutation invariant**.

“The cat sat on the mat”

Token positions: 0, 1, 2, 3, 4, 5

“mat the on sat cat The”

Shuffled positions: 5, 4, 3, 2, 1, 0

For the attention formula $\text{Attention}(Q, K, V)$, these two sentences produce **identical representations** if we ignore token positions!

Solution: Add position information to embeddings.

Sinusoidal Positional Encoding Formula:

$$\text{PE}_{(\text{pos}, 2i)} = \sin \left(\frac{\text{pos}}{10000^{\left(2 \frac{i}{d_{\text{model}}}\right)}} \right)$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos \left(\frac{\text{pos}}{10000^{\left(2 \frac{i}{d_{\text{model}}}\right)}} \right)$$

Parameters:

- **pos**: Position of the token in the sequence (0, 1, 2, ...)
- **i**: Dimension index $\left(0, 1, 2, \dots, \frac{d_{\text{model}}}{2} - 1\right)$

Combined with token embedding: $X = E + \text{PE}$ (element-wise addition)

Each dimension encodes a different frequency:

$$\omega_i = 1/10000^{(2i/d_{\text{model}})}$$

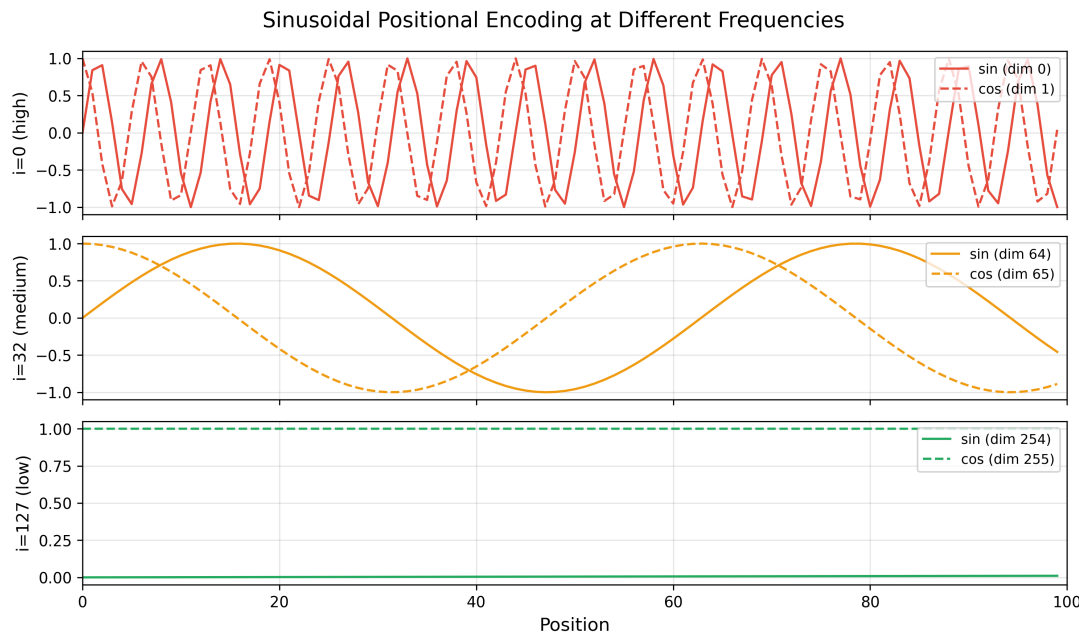
- Dimension 0 ($i = 0$): $\omega = 1 \rightarrow$ one cycle per position
- Dimension 64 ($i = 32$): $\omega \approx 0.1 \rightarrow$ slower oscillation

Result: Each position gets a unique “signature” across all dimensions.

Analogy: GPS coordinates

- Latitude + Longitude = unique location
- Multiple sinusoids at different frequencies = unique position

Visualizing Different Frequencies

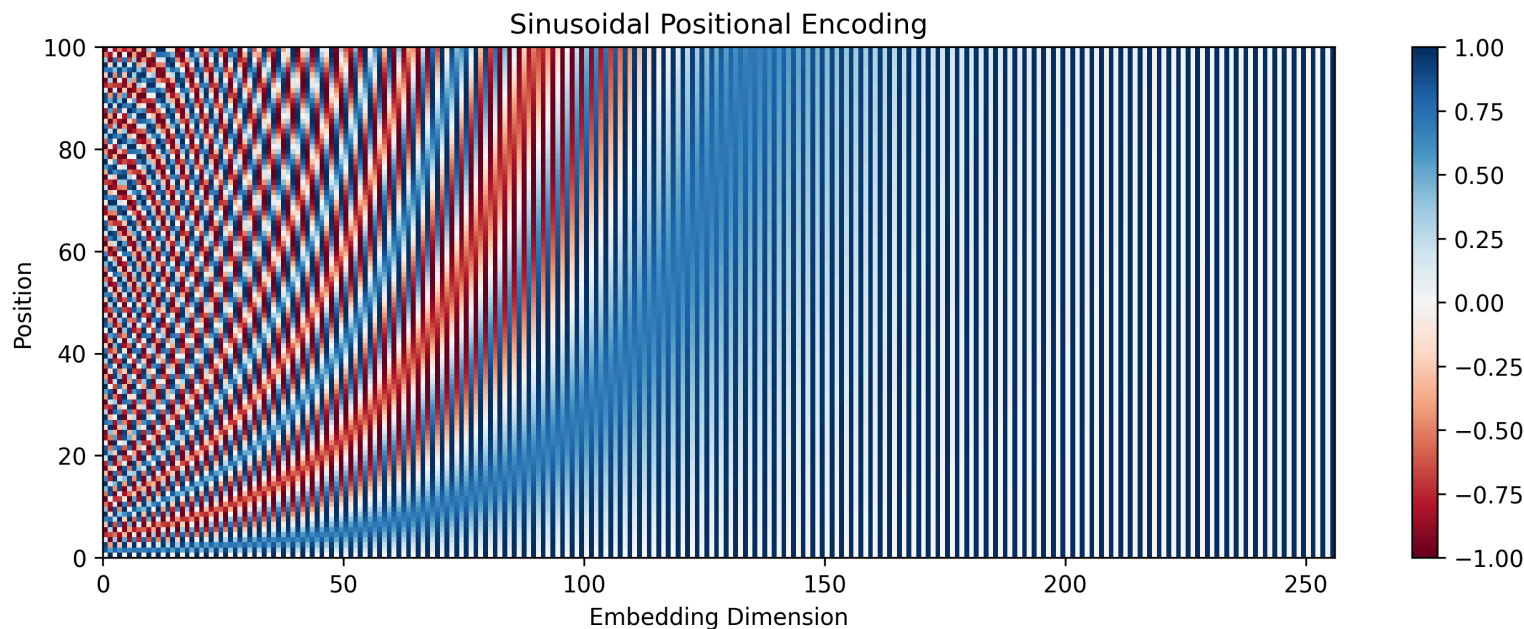


- **High frequency** ($i = 0$): Rapid oscillations $\rightarrow$ distinguishes adjacent positions
- **Medium frequency** ($i = 32$): Moderate oscillations $\rightarrow$ phrase-level patterns
- **Low frequency** ($i = 127$): Slow oscillations $\rightarrow$ distinguishes distant positions

The Full Positional Encoding Matrix



南方科技大学



- **Each row:** A unique vector for that position
- **Left side (low i):** High-frequency ripples $\rightarrow$ encode local relative positions
- **Right side (high i):** Low-frequency waves $\rightarrow$ encode long-range dependencies

Key Property: Relative Position Encoding



南方科技大学

Goal: Enable learning relative positions, not just absolute positions.

Desired property: $\text{PE}_{\text{pos}+k} = \text{PE}_{\text{pos}} \cdot M_k$ where M_k depends only on offset k

$$\begin{aligned}\text{PE}_{\text{pos}+k, 2i} &= \sin((\text{pos} + k)\omega_i) \\ &= \sin(\text{pos } \omega_i) \cdot \cos(k\omega_i) + \cos(\text{pos } \omega_i) \cdot \sin(k\omega_i) \\ &= \text{PE}_{\text{pos}, 2i} \cdot \cos(k\omega_i) + \text{PE}_{\text{pos}, 2i+1} \cdot \sin(k\omega_i)\end{aligned}$$

$$\text{For each pair } (2i, 2i + 1): M_k^{2i} = \begin{pmatrix} \cos(k\omega_i) & \sin(k\omega_i) \\ -\sin(k\omega_i) & \cos(k\omega_i) \end{pmatrix}$$

Implication: Attention can learn “**Relative**” patterns like “word 3 positions ahead”.

Key Property: Extrapolation & Efficiency



南方科技大学

While learnable embeddings $(E + W_{\text{pos}})$ are common, the sinusoidal version offers unique advantages:

1. Extrapolation

Can generalize to sequence lengths **longer** than those seen during training.

2. Efficiency

Parameter-free; memory-efficient and prevents overfitting on specific indices.

Examples of Extrapolation:

- **Sequence Length:** Train on a max length of 512 tokens, but infer on a document of 2048 tokens.
- **Continuous Functions:** The $\frac{\sin}{\cos}$ mathematical waves extend infinitely, providing unique vectors for any unseen position index $p > 512$.

```
1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_len=5000):
3         super().__init__()
4         pe = torch.zeros(max_len, d_model)
5         position = torch.arange(0, max_len).unsqueeze(1)
6         div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
7         pe[:, 0::2] = torch.sin(position * div_term) # even indices
8         pe[:, 1::2] = torch.cos(position * div_term) # odd indices
9         self.register_buffer('pe', pe) # not a parameter, but saved
10    def forward(self, x):
11        return x + self.pe[:, x.size(1)] # x: (Batch, Seq_Len, d_model)
```

Python

Key points:

- register_buffer: Saves tensor with model (so do not compute every forward pass), but not trainable.
- Slicing `pe[:, x.size(1)]`: Supports variable sequence lengths.

Modern Choice: Rotary Position Embedding (RoPE) (used in Llama, Qwen, DeepSeek) — instead of **adding** a position vector, it **rotates** query and key vectors:

$$(R_m q)^T (R_n k) = q^T R_{m-n} k$$

Ensuring that attention scores depend only on the **relative distance** $m - n$.

Why RoPE?

- **Relative Dist:** Purely depends on $|m - n|$, enabling natural distance awareness.
- **Scaling:** Highly compatible with context length extension (e.g., 128k+).
- **Stability:** Preserves vector norms during positioning, preventing embedding “drift.”

Residual Connections & Layer Normalization

Enable stable training of deep Transformer stacks with skip connections and Pre-LN.

The Challenge: Training Deep Networks



南方科技大学

Why do we need deep networks?

- More layers $\rightarrow$ more capacity to learn complex patterns.
- **Scaling laws:** Deeper/wider models perform better.

Modern LLMs have $L > 50$ layers (e.g., Original Transformer: only 6 layers, GPT-3: 96, Qwen3.5: 60, but wider).

Problem: Gradients are multiplied L times via chain rule: $\prod_{i=1}^L (\partial f_i) / (\partial f_{i-1})$

1. Vanishing Gradients

Each factor $< 1 \rightarrow$ product $\rightarrow 0$

2. Exploding Gradients

Each factor $> 1 \rightarrow$ product $\rightarrow \infty$

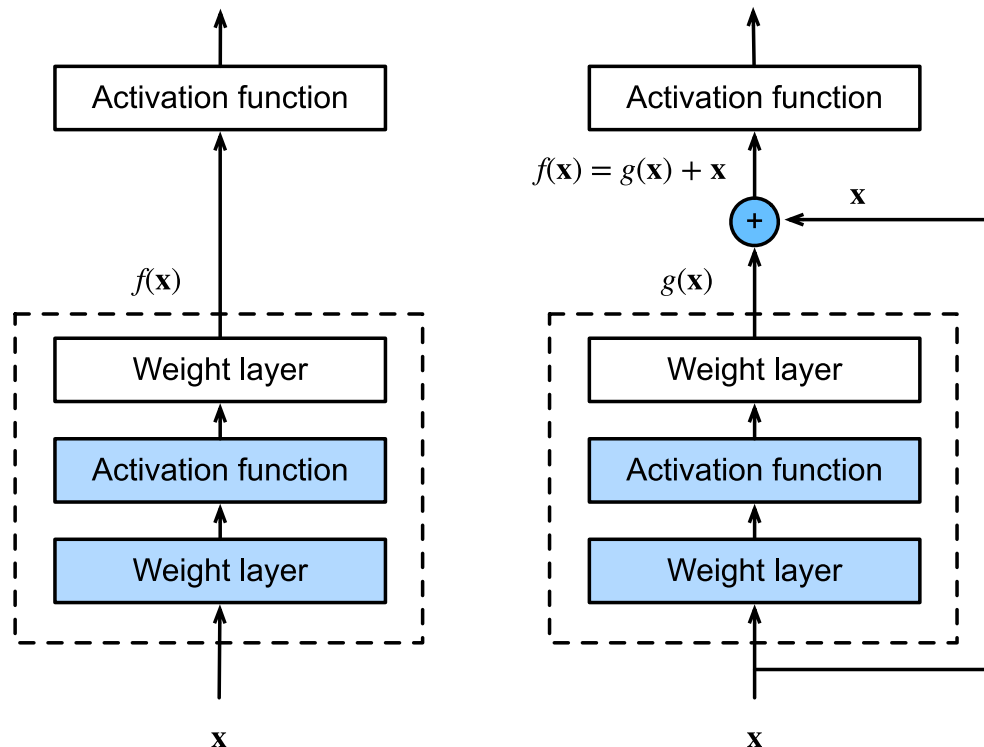
Solution: Use **Residual Connections** (Skip Connections), and **Normalization**.

Core Idea: Learn the residual instead of the full mapping.

$$\text{Output } f(x) = x + g(x)$$

Benefits:

- Gradient flows directly through identity path x .
- Enables training very deep networks (100+ layers).



Another deep network challenge: Input distributions shift across layers during training. **Normalization can stabilize this.**

Why not Batch Normalization (BN)? (widely used before Transformer)

1. **Transformer Input:** Sequences have **variable lengths** within a batch.
2. **Transformer Training:** Batch statistics are unstable for small batch sizes.

Layer Normalization: Normalize across **features for each sample** independently.

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$$

- μ, σ : Mean and std over feature dimension
- γ, β : Learnable scale and shift



Modern Variant: RMSNorm (used in Llama, Qwen, DeepSeek) — drops the mean centering, normalizes by root-mean-square only:

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}}$$

Simpler & faster, with comparable performance.

Why RMSNorm?

- It is observed that mean centering $(x - \mu)$ in LayerNorm adds computation but contributes little to training stability.
- RMSNorm removes it, saving compute with no obvious accuracy loss.

Post-LN (Original Transformer)

$$x = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Pre-LN (GPT-3, Llama, etc.)

$$x = x + \text{Sublayer}(\text{LayerNorm}(x))$$

Gradient flow via chain rule:

- Post-LN: $\frac{\partial x_{\text{out}}}{\partial x_{\text{in}}}$ passes through LayerNorm — skip path **blocked**.
- Pre-LN: $\frac{\partial x_{\text{out}}}{\partial x_{\text{in}}} = 1 + \text{Other Term}$ — skip path **preserved**.

Why Pre-LN is more stable: The residual path remains **uninterrupted** — gradients can skip all sublayers and flow directly to earlier layers.

PyTorch: Pre-LN Residual Block



```
1 class PreLNResidualBlock(nn.Module):
2     def __init__(self, d_model):
3         super().__init__()
4         self.norm = nn.LayerNorm(d_model) # or use nn.RMSNorm(d_model)
5
6     def forward(self, x, sublayer_output):
7         # Pre-LN pattern: norm -> sublayer -> add
8         return x + sublayer_output(self.norm(x))
```

Python

Usage in Transformer Block:

```
1 # Inside TransformerEncoderBlock
2 x = self.residual1(x, lambda h: self.self_attn(h, h, h)[0])
3 x = self.residual2(x, lambda h: self.ffn(h))
```

Python

Key pattern: The lambda wraps the sublayer so it can be called with normalized input.

Cross-Attention

Bridge encoder and decoder by attending across two different sequences.

Recap: Self-Attention



南方科技大学

In **Self-Attention**, Query, Key, and Value all come from the **same sequence**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Input Sequence $X \longrightarrow$ Linear Projections $\longrightarrow Q, K, V$

- Every position attends to every other position.
- Captures relationships **within** a single sequence.

Cross-Attention: Bridging Two Sequences



In **Cross-Attention**, Query comes from one sequence, while Key and Value come from another.

Self-Attention

$$X \rightarrow Q, K, V$$

All from same sequence

Cross-Attention

$$Y \rightarrow Q \text{ (Query)}$$

$$X \rightarrow K, V \text{ (Key, Value)}$$

From different sequences

Example: Chinese $\rightarrow$ English Translation

Input Sequence (Source: 中文)

“我” “爱” “你”

$\rightarrow K, V \rightarrow$

Output Sequence (Target: English)

“I” “love” “you”

Result: “love” attends strongly to “爱” (the relevant source word in other sequence).


```
1 # Option 1: nn.MultiheadAttention
2 self.cross_attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
3 # Option 2: Your Lab 5 MHA implementation
4 self.cross_attn = MultiHeadAttention(d_model, num_heads)
5 # In forward:
6 attn_output, _ = self.cross_attn(
7     decoder_hidden,      # from decoder
8     encoder_output,      # from encoder
9     encoder_output       # from encoder
10 )
```

 Python

Tensor shapes:

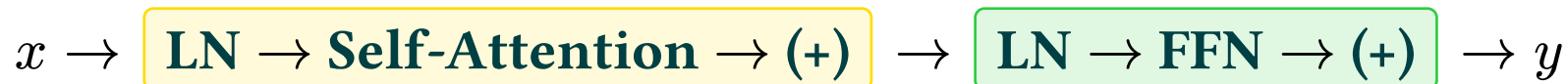
- Q : shape= $(B, T_{\text{dec}}, d_{\text{model}})$ — from decoder (target sequence)
- K, V : shape= $(B, T_{\text{enc}}, d_{\text{model}})$ — from encoder (source sequence)

Key difference from self-attention: query and key/value from **different** sources.

Transformer Blocks

Assemble sub-layers into Encoder, Decoder-Only, and Encoder-Decoder blocks.

One **Encoder Block** = two sub-layers with Pre-LN residual connections:



Self-Attention

Every position attends to **all** positions
Self-Attention

Feed-Forward Network (FFN)

$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$

(+) represents a Residual Connection

Key properties:

- Input/Output shape: (B, T, d_{model}) — identical shape enables stacking N blocks
- No masking — each token sees the **entire** sequence

Representative models: **BERT** (for understanding, classification).

Modern Choice: SwiGLU (used in Llama, Qwen, DeepSeek) — a Gated Linear Unit (GLU) variant that replaces ReLU in the FFN:

$$\text{SwiGLU}(x, W, V, b, c) = \text{Swish}(xW + b) \cdot (xV + c)$$

Where $\text{Swish}(x) = x \cdot \sigma(\beta x)$. It is the current state-of-the-art for LLM FFNs.

Why SwiGLU?

- **Gating Mechanism:** Allows the network to learn which information to pass through more effectively than simple ReLU.
- **Smoothness:** Swish is a smooth, non-monotonic function, which often results in better optimization and performance in deep models.

PyTorch: Encoder Block



```
1 class TransformerEncoderBlock(nn.Module):
2     def __init__(self, d_model, num_heads, d_ff):
3         super().__init__()
4         self.norm1 = nn.LayerNorm(d_model)
5         self.norm2 = nn.LayerNorm(d_model)
6         self.self_attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
7         self.ffn = nn.Sequential(
8             nn.Linear(d_model, d_ff),
9             nn.ReLU(),
10            nn.Linear(d_ff, d_model)
11        )
12    def forward(self, x):
13        norm_x = self.norm1(x) # Pre-LN
14        attn_out, _ = self.self_attn(norm_x, norm_x, norm_x) # Self-Attention
15        x = x + attn_out # Residual Connection
16        x = x + self.ffn(self.norm2(x)) # Pre-LN FFN
17    return x
```

Python

The Decoder Block (Decoder-Only)



A **Decoder Block** has the same structure as an Encoder, but uses **Causal Masking**:

$$x \rightarrow \boxed{\text{LN} \rightarrow \text{Masked Self-Attn} \rightarrow (+)} \rightarrow \boxed{\text{LN} \rightarrow \text{FFN} \rightarrow (+)} \rightarrow y$$

Causal Masking

Token at position t can only attend to positions $\leq t$

Prevents “seeing the future” during generation
(Recap from Lab 5)

FFN

Same position-wise transform

Autoregressive Generation

Generates tokens one-by-one

Each new token is conditioned on all previous tokens

Same building blocks as Encoder — the only difference is the causal attention mask.

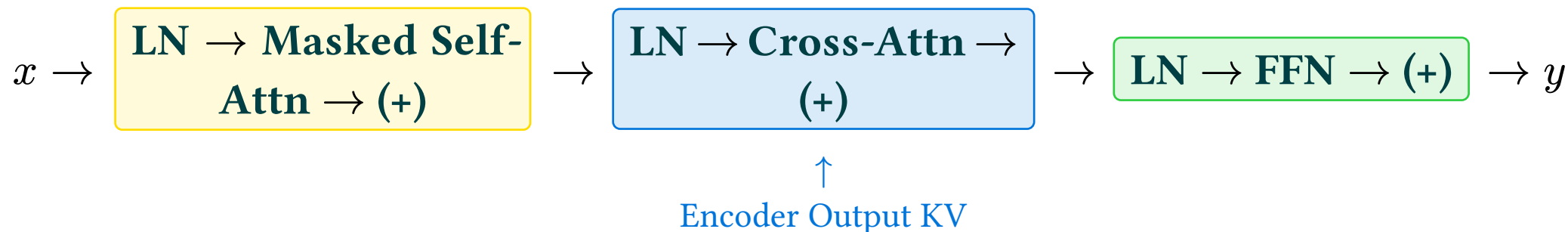
Representative models: **GPT, Llama, Qwen** (for text generation, chat).

The Encoder-Decoder Decoder Block



南方科技大学

In an Encoder-Decoder model, the Decoder Block gains a **Cross-Attention** sub-layer:



Masked Self-Attn

Causal mask on target sequence

Cross-Attn (No Mask)

Q from decoder
 K, V from encoder output

FFN

Same position-wise transform

Representative models: **Original Transformer**, **T5** (for translation, summarization).

The Full Encoder-Decoder Architecture



南方科技大学

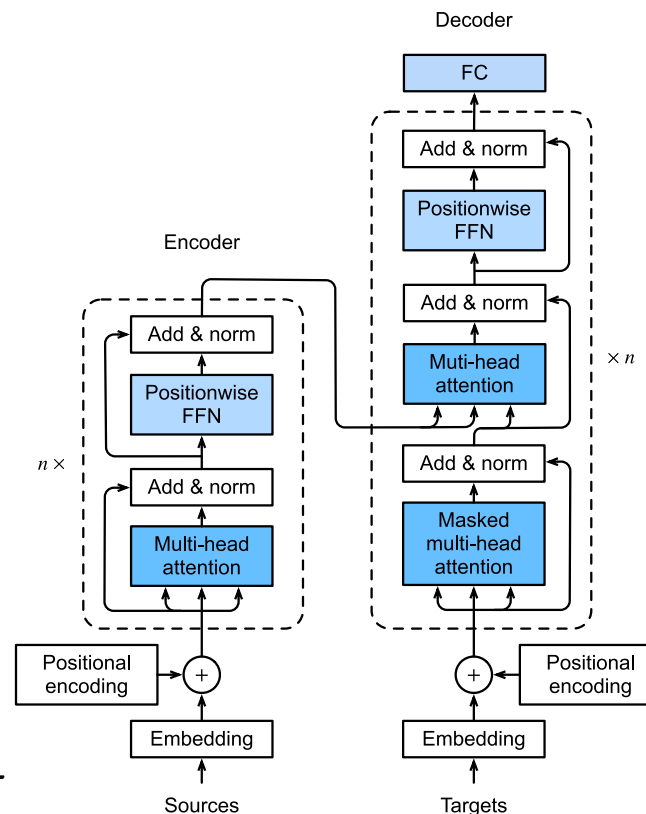
Encoder

1. Input Embedding + Positional Encoding
2. Stack of N Encoder Blocks
 - Each: Self-Attn + FFN (with Pre-LN)

Decoder

1. Output Embedding + Positional Encoding
2. Stack of N Decoder Blocks
 - Each: Masked Self-Attn + Cross-Attn + FFN
3. Final Linear + Softmax

Modern trend: Decoder-only dominates because it's simpler to scale and fine-tune.



Transformer Architecture

Lab 6 Exercises

Build a complete Transformer step by step: Encoder Block $\rightarrow$ Decoder Block $\rightarrow$ Full Assembly.

Task 1: Encoder Block (`lab6_encoder.py`) [30 points]

- Implement `TransformerEncoderBlock` (`__init__` + `forward`)
- Build the 2-sublayer Pre-LN residual pattern from scratch
- Use `nn.MultiheadAttention` for Self-Attention

Task 2: Decoder Block (`lab6_decoder.py`) [30 points]

- Implement `TransformerDecoderBlock` (`__init__` + `forward`)
- Extend the Encoder pattern: add **Cross-Attention** as a 3rd sub-layer
- Cross-Attention: Query from decoder, Key/Value from encoder output

Task 3: Full Transformer (`lab6_transformer.py`) [40 points]

- Implement `__init__`, `encode()`, `decode()`, and `forward()`
- Wire embeddings, positional encoding, encoder/decoder stacks, output projection

Task 4: Implementing RoPE [Bonus]

- Construct the block-diagonal rotation matrix in `get_rope_matrix()`
- Apply the rotation using pure matrix multiplication in `apply_rope_matrix()`
- Extract matrix diagonals and plot the rotation heatmaps (Provided)
- **Discussion:** Is this implementation low-efficient? Can you think of a more efficient way to implement RoPE?

RoPE Implementation Requirements

1. Formula (θ_i computation):

The base angle for the i -th dimension pair is $\theta_i = 10000^{-2\frac{i}{d}}$, where $i \in [0, \frac{d}{2} - 1]$. For a token sequence at position m , the scaled angle is $m\theta_i$.

2. Matrix Construction:

Construct the true $(d_{\text{model}}, d_{\text{model}})$ block-diagonal rotation matrix R_m where each 2×2 block is:

$$R_{m,i} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$

The final matrix should look like:

$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

- `get_rope_matrix`: Compute θ , multiply by all positions m , calculate $\cos / \sin$, then layout the $[\text{max_len}, \text{d_model}, \text{d_model}]$ tensor filled with these 2×2 blocks along the diagonal.
- `apply_rope_matrix`: Use `torch.matmul` to multiply the queries/keys: $q_{\text{rope}} = qR$.

Hints for Implementation



For Task 1 (Encoder Block):

```
1 # Pre-LN residual pattern (same for each sub-layer):
2 norm_x = self.norm1(x)
3 attn_out, _ = self.self_attn(norm_x, norm_x, norm_x)
4 x = x + attn_out # residual connection
```

 Python

For Task 2 (Decoder Block):

```
1 # Sub-layer 2: Cross-Attention – note Q vs K/V sources
2 norm_x = self.norm2(x)
3 attn_out, _ = self.cross_attn(query=norm_x, key=enc_output, value=enc_output)
4 x = x + attn_out
```

 Python

Everything About Transformers

- A comprehensive visual guide to Transformer architecture.
- <https://www.krupadave.com/articles/everything-about-transformers>

Rotary Position Embedding (RoPE)

- Course Material: Lecture 3 (理论课课件)
- EleutherAI Blog: [Rotary Embeddings: A Relative Revolution](#)

FlashAttention / Online Softmax (Covered in Lecture 6)¹

- A CUDA kernel implementation of FlashAttention for learning purposes:
- [LeetCUDA: FlashAttention Kernel](#) and [LeetCUDA: Online Softmax Kernel](#)

¹Involves low-level CUDA kernel programming (Tiling, CUTLASS). Not part of COE 201 lab requirements — students with GPU programming experience are welcome to explore independently.

Let's build a Transformer!